pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

Changes in bp and Percent

WEEK 2 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note: how far each series moved

The Desk Lead's Note

Week 2 measures the dataset. The morning note opens with how far each series moved: on the day, over the week, over the month, and since the end of last year. Give me that for the 2, 5, 10 and 30 Yr and the two synthetic spreads as of 2 October, each in its right unit and tied to Excel. A market holiday must never show up as a zero move. Then show me what September's move in the 5 Yr did to a bond's price. Last, which days of 2026 moved the 10 Yr most? Describe; no views.

The two files are notebook 05's: Treasury yields in percent, and synthetic spreads in bp, invented for this course, not ICE, Moody's, or any index. Today reads only these files, so there is no key cell and no live cell.

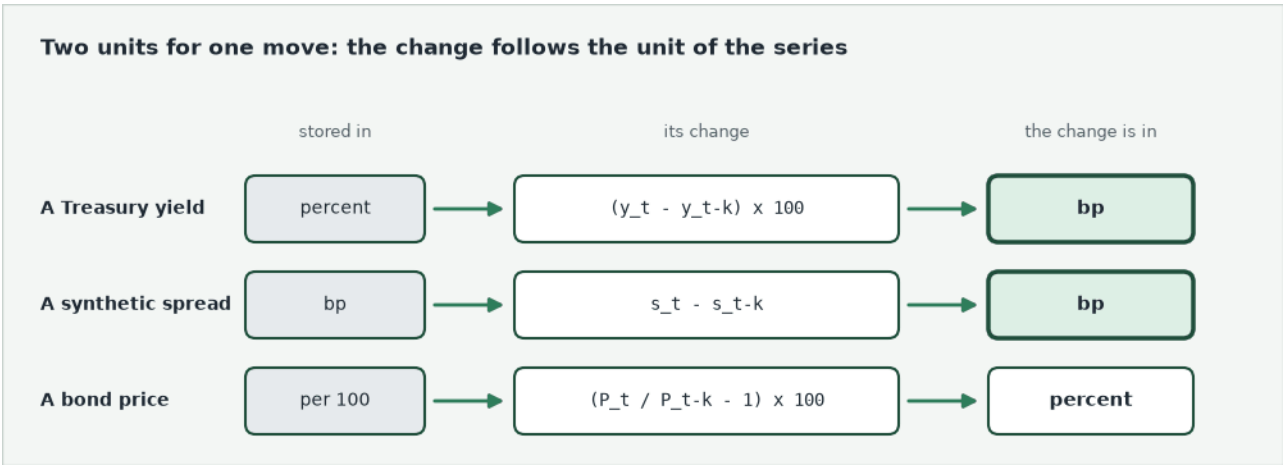
```
Work folder pfi_course3_06_k4w7q2xr: 4 data files, Course 2's bondmath.py, marketdata.py as notebook 05 left it, 5 commits, tagged week1
....................   [100%]
20 passed in 0.50s
```

The Two Files

```
treasury = pd.read_csv('treasury_par_yields.csv', parse_dates=['date'], index_col='date',
                       float_precision='round_trip')
spreads = pd.read_csv('course3_synthetic_spreads.csv', parse_dates=['date'], index_col='date',
                      float_precision='round_trip')
ten = treasury['10 Yr']
ig = spreads['ig_spread_bp']
```

Yields arrive in percent and spreads in bp: the unit of a move starts from the unit of the series.

6.1 Yields and spreads change in bp, prices in percent



A yield and a spread are rates, so a move is a difference of rates, in bp. A price is money, so a move is a share of where it started.

The Last Day, by Hand

Q. How far did the 10 Yr and the synthetic IG spread move on 2 October?

```
ten_move_bp = (ten.iloc[-1] - ten.iloc[-2]) * 100
ig_move_bp = ig.iloc[-1] - ig.iloc[-2]
print(f'10 Yr: {ten.iloc[-2]:.2f} to {ten.iloc[-1]:.2f} percent, {ten_move_bp:+.1f} bp')
print(f'IG spread (synthetic): {ig.iloc[-2]:.1f} to {ig.iloc[-1]:.1f} bp, {ig_move_bp:+.1f} bp')
10 Yr: 5.24 to 5.28 percent, +4.0 bp
IG spread (synthetic): 98.0 to 97.0 bp, -1.0 bp
```

A yield's move is a difference times 100, a spread's a plain difference, both in bp. Next, one function for both.

"+" in a format code always shows the sign, as in Course 2's Price-Yield notebook. Write k for the number of rows back: a change is always from an earlier row, never from a day with no row.

6.2 change_bp: rows back, not days back

The hand sum becomes a function. It needs one fact beyond the series: is it in percent? Q. Read the docstring, then append "change_bp".

```
%%writefile -a marketdata.py
def change_bp(series: pd.Series, periods: int = 1, percent_units: bool = True) -> pd.Series:
    """Change over `periods` observations, in basis points.
    Units: the result is in bp. percent_units=True for a series in percent (a yield): (x_t - x_{t-k}) x 100.
    percent_units=False for a series already in bp (a spread): x_t - x_{t-k}.
    Convention: k rows back, whatever the calendar gap (the Tuesday after a Monday holiday is a change from
    Friday). The first `periods` values are NaN. Never a percent change of a yield or spread level.
    Excel: =(B3-B2)*100 for a yield in percent.
    Raises ValueError if periods is below 1, the series has missing values, or its index is not unique
    ascending dates.
    """
    _check_series(series, "series")
    if periods < 1:
        raise ValueError(f"periods must be at least 1, not {periods}")
    difference = series - series.shift(periods)
    return difference * 100 if percent_units else difference
```

One and five rows back, and the week of Labor Day

One and Five Observations

Q. The 10 Yr's level with its 1-row and 5-row changes, for the last six days.

```
importlib.reload(md)
moves = pd.DataFrame({'level_pct': ten, 'chg_1_bp': md.change_bp(ten),
                      'chg_5_bp': md.change_bp(ten, periods=5)})
moves.tail(6).round(2)
```

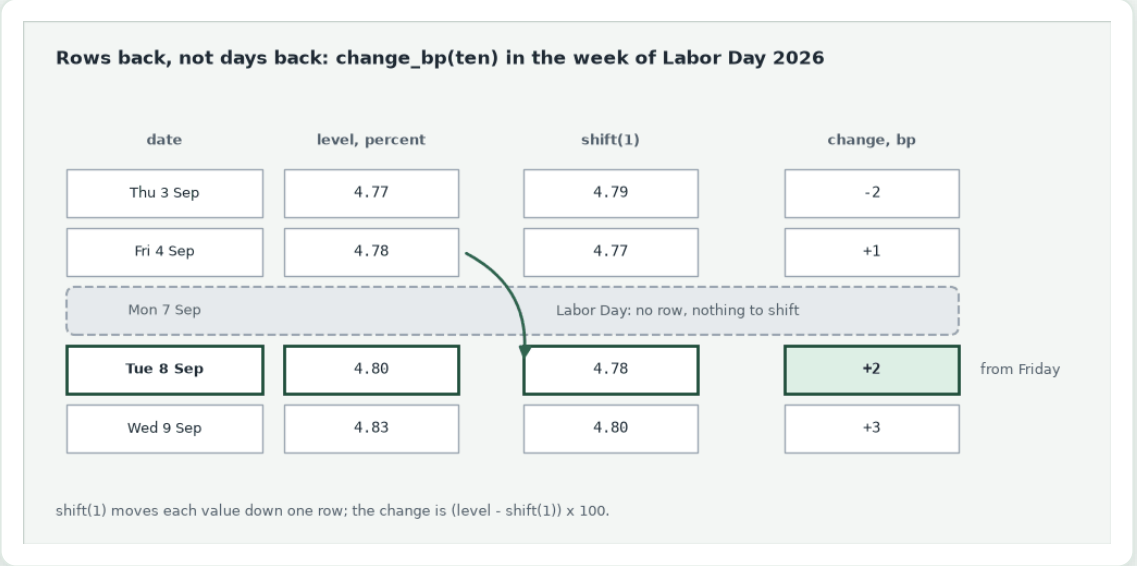
date	level_pct	chg_1_bp	chg_5_bp
2026-09-25	5.17	-1.0	16.0
2026-09-28	5.24	7.0	28.0
2026-09-29	5.26	2.0	30.0
2026-09-30	5.29	3.0	18.0
2026-10-01	5.24	-5.0	6.0
2026-10-02	5.28	4.0	11.0

"series.shift(periods)" is notebook 03's ".shift()", moved "periods" rows; "_check_series" is notebook 04's.

The Week of Labor Day

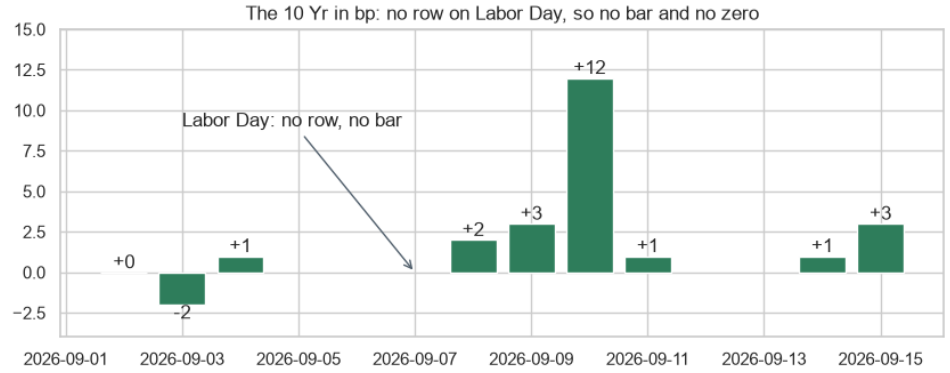
Q. Monday 7 September has no row. What does Tuesday's change compare with?

```
moves.loc['2026-09-03':'2026-09-09'].round(2)
```



Tuesday is +2 bp from Friday's 4.78: a holiday is skipped, never a zero change.

Two weeks of September, and every row against Excel



Wednesday 2 September's +0 is a real row that did not move; Labor Day is no row at all.

Q. Does "change_bp" equal Excel's daily change on every row of 2026?

```
reference = pd.read_csv('course3_excel_reference.csv', parse_dates=['date'], index_col='date',
                        float_precision='round_trip')
gap = (md.change_bp(ten).loc[reference.index] - reference['chg_10y_bp']).abs().max()
print(f'{len(reference)} rows of 2026, largest gap {gap:.1e} bp')
190 rows of 2026, largest gap 0.0e+00 bp
```

Excel side by side. Dates in A, the 10 Yr in B, from 2025-01-02 in row 2.

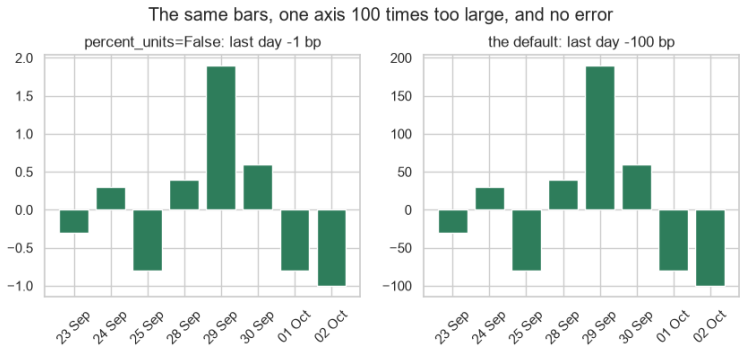
Job	Python	Excel	Result
Daily change in bp, C3 filled down	`md.change_bp(ten)`	`=(B3-B2)*100`	Equal on all 190 rows of 2026

"change_bp" counts rows, so a holiday is skipped, never a zero, and it equals Excel on every row of 2026. Next, the inputs it cannot fix by itself.

6.3 A spread read as a yield

Two inputs can still go wrong, and a price needs a function of its own. Q. What if the spread's unit is left at the default?

```
ig_bp = md.change_bp(ig, percent_units=False)
ig_as_yield = md.change_bp(ig)
print(f'Last day: {ig_bp.iloc[-1]:+.1f} bp, or {ig_as_yield.iloc[-1]:+.1f} read as a yield')
Last day: -1.0 bp, or -100.0 read as a yield
```



The Mars Climate Orbiter

Course 1 told how NASA lost the Mars Climate Orbiter in 1999. Thruster data was supplied in English units where the trajectory models expected metric, and the spacecraft arrived about 170 kilometers lower than planned.

Source: Mars Climate Orbiter Mishap Investigation Board, Phase I Report, November 10, 1999, pages 6 and 7. https://llis.nasa.gov/llis_lib/pdf/1009464main1_0641-mr.pdf

Excel side by side. The synthetic IG spread in D, the last row 440.

Job	Python	Excel	Result
The spread's last change	`md.change_bp(ig, percent_units=False)`	`=D440-D439`	-1
The same, read as a yield	`md.change_bp(ig)`	`=(D440-D439)*100`	-100

A tenor with empty rows, and the price's function

A Tenor With Empty Rows

Q. The 4 Mo starts in October 2022. Can "change_bp" take it as it is?

```
md.change_bp(treasury['4 Mo'])
ValueError: series has 1951 missing values; drop them first with .dropna()
```

```
four_month = treasury['4 Mo'].dropna()
print(f'{len(four_month):,} rows from {four_month.index[0]:%Y-%m-%d}, '
      f'last change {md.change_bp(four_month).iloc[-1]:+.1f} bp')
989 rows from 2022-10-19, last change +0.0 bp
```

Drop, never fill, before a change: a filled blank is a zero move the market never made.

The Price's Function: return_pct

```
%%writefile -a marketdata.py
def return_pct(series: pd.Series, periods: int = 1) -> pd.Series:
    """Simple return over `periods` observations, in percent, for a price series.
    Units: prices in, percent out. Convention: (P_t / P_{t-k} - 1) x 100, k rows back; no log returns.
    The first `periods` values are NaN. Excel: =(C3/C2-1)*100
    Raises ValueError if periods is below 1, the series has missing values, or its index is not unique
    ascending dates.
    """
    _check_series(series, "series")
    if periods < 1:
        raise ValueError(f"periods must be at least 1, not {periods}")
    return (series / series.shift(periods) - 1) * 100
```

Up 1 Percent, Down 1 Percent

```
importlib.reload(md)
hand_prices = pd.Series([100.0, 101.0, 99.99],
                        index=pd.to_datetime(['2026-09-28', '2026-09-29', '2026-09-30']))
print(md.return_pct(hand_prices).round(6).tolist())
[nan, 1.0, -1.0]
```

Up 1 percent, down 1 percent, and the price is 99.99, not back at 100: each return is a share of its own start.

Three tests

```
%%writefile -a test_marketdata.py
def test_change_bp_hand_series():
    dates = pd.to_datetime(["2026-09-03", "2026-09-04", "2026-09-08"])
    yields = pd.Series([4.00, 4.10, 4.05], index=dates)
    changes = md.change_bp(yields)
    assert math.isnan(changes.iloc[0])
    assert changes.iloc[1] == pytest.approx(10.0, abs=1e-9)
    assert changes.loc["2026-09-08"] == pytest.approx(-5.0, abs=1e-9)
    assert md.change_bp(yields, periods=2).iloc[2] == pytest.approx(5.0, abs=1e-9)
def test_change_bp_spread_units():
    spreads = pd.Series([98.8, 101.3], index=pd.to_datetime(["2026-09-29", "2026-09-30"]))
    assert md.change_bp(spreads, percent_units=False).iloc[1] == pytest.approx(2.5, abs=1e-9)
def test_return_pct_hand_series():
    prices = pd.Series([100.0, 101.0, 99.99], index=pd.to_datetime(["2026-09-28", "2026-09-29", "2026-09-30"]))
    returns = md.return_pct(prices)
    assert returns.iloc[1] == pytest.approx(1.0, abs=1e-12)
    assert returns.iloc[2] == pytest.approx(-1.0, abs=1e-12)
```

```
.....................   [100%]
23 passed in 0.50s
```

"math.isnan" is notebook 02's: the first change has nothing before it.

Pass "percent_units=False" for a spread, drop blank rows first, and use "return_pct" for a price. Now the trap the unit argument guards against.

6.4 Why a percent change of a yield misleads

pandas has "pct_change()", and a spreadsheet has "=B2/B1-1". Both are right for a price.

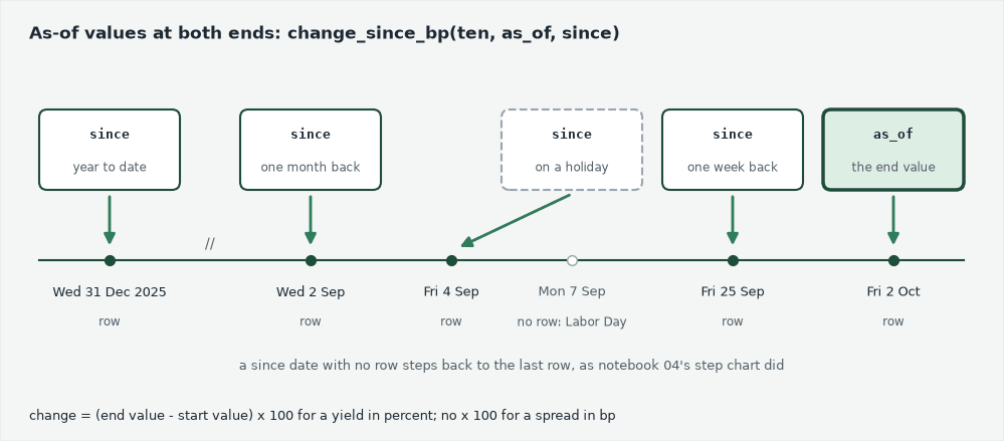
Excel side by side. 4.00 in B1, 4.10 in B2.

Job	Python	Excel
The move in bp	`(4.10 - 4.00) * 100`	`=(B2-B1)*100`
The percent of the yield	`pct_change()`	`=B2/B1-1, percent format`

Course 2's DV01 is per bp, so the bp figure is the one a price answers to.

A relative move of a spread, when someone asks for one, is labelled "relative change, percent".

6.5 Changes since a date: change_since_bp



"Up 11 bp on the week" is a change since a date, and that date may have no row. Each end is notebook 04's as-of value, the step its chart drew.

Two "value_as_of" calls and a subtraction. A week back is "pd.Timedelta(days=7)", as in Course 1; a month back is notebook 05's "pd.DateOffset(months=1)".

```
%%writefile -a marketdata.py
def change_since_bp(series: pd.Series, as_of, since, percent_units: bool = True) -> float:
    """Change from the as-of value at `since` to the as-of value at `as_of`, in basis points.
    Units: the result is in bp; percent_units as change_bp. Convention: each end is the last observation
    on or before its date (value_as_of), so a `since` on a holiday or weekend uses the market day before it.
    One week back is as_of - 7 days; one month back is as_of - pd.DateOffset(months=1).
    Excel: =(XLOOKUP(as_of,A:A,B:B,,-1)-XLOOKUP(since,A:A,B:B,,-1))*100
    Raises ValueError if since is after as_of or before the first observation.
    """
    if pd.Timestamp(since) > pd.Timestamp(as_of):
        raise ValueError(f"since ({pd.Timestamp(since).date()}) must not be after as_of ({pd.Timestamp(as_of).date()})")
    _, value_now = value_as_of(series, as_of)
    _, value_then = value_as_of(series, since)
    difference = value_now - value_then
    return difference * 100 if percent_units else difference
```

Week, month, year to date, and a start with no row

Q. How far has the 10 Yr moved as of 2 October, and from which rows?

```
importlib.reload(md)
as_of = pd.Timestamp('2026-10-02')
starts = {'week': as_of - pd.Timedelta(days=7), 'month': as_of - pd.DateOffset(months=1),
          'year to date': pd.Timestamp('2025-12-31')}
for label, since in starts.items():
    used, value = md.value_as_of(ten, since)
    print(f'{label:<12} from {used:%Y-%m-%d} at {value:.2f}: '
          f'{md.change_since_bp(ten, as_of, since):+.0f} bp')
week   from 2026-09-25 at 5.17: +11 bp
month   from 2026-09-02 at 4.79: +49 bp
year to date from 2025-12-31 at 4.18: +110 bp
```

A Start Date With No Row

```
used, value = md.value_as_of(ten, '2026-09-07')
holiday = md.change_since_bp(ten, '2026-09-30', '2026-09-07')
print(f'Since 2026-09-07: the row of {used:%Y-%m-%d} at {value:.2f}, '
      f'{holiday:+.0f} bp to 2026-09-30')
Since 2026-09-07: the row of 2026-09-04 at 4.78, +51 bp to 2026-09-30
```

Excel side by side. The same sheet, 2026 from row 251.

Job	Python	Excel	Result
One month, filled down	`md.change_since_bp(ten, day, day - pd.DateOffset(months=1))`	`=(B251-XLOOKUP(EDATE(A251,-1),A:A,B:B,,-1))*100`	Equal on all 190 rows
One week, filled down	`md.change_since_bp(ten, day, day - pd.Timedelta(days=7))`	`=(B251-XLOOKUP(A251-7,A:A,B:B,,-1))*100`	Equal on all 190 rows
Year to date	`md.change_since_bp(ten, as_of, '2025-12-31')`	`=(XLOOKUP(DATE(2026,10,2),A:A,B:B,,-1)`	110.00000000000006
		`-XLOOKUP(DATE(2025,12,31),A:A,B:B,,-1))*100`	
Since Labor Day	`holiday`	`the same, with DATE(2026,9,30) and DATE(2026,9,7)`	50.99999999999998
A month before 31 March	`pd.DateOffset(months=1)`	`=EDATE(DATE(2026,3,31),-1)`	2026-02-28 in both

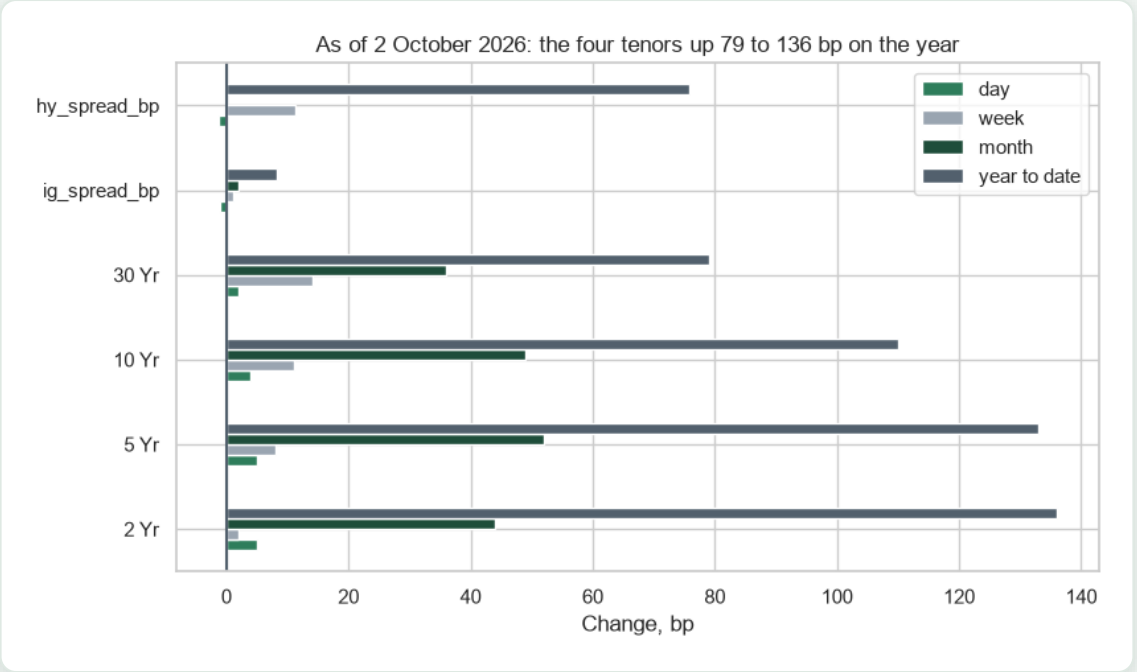
The morning note's table, all in bp

Q. Every series the note names, on the day, the week, the month and the year, all in bp.

```
market = treasury[['2 Yr', '5 Yr', '10 Yr', '30 Yr']].join(spreads)
periods = {'day': pd.Timestamp('2026-10-01')} | starts
```

	day	week	month	year to date
2 Yr	+5.0	+2.0	+44.0	+136.0
5 Yr	+5.0	+8.0	+52.0	+133.0
10 Yr	+4.0	+11.0	+49.0	+110.0
30 Yr	+2.0	+14.0	+36.0	+79.0
ig_spread_bp	-1.0	+1.1	+1.9	+8.2
hy_spread_bp	-1.3	+11.2	+0.1	+75.7

The table describes the file as of 2 October and says nothing of what comes next.



Two tests against Excel

```
%%writefile -a test_marketdata.py
def read_reference() -> pd.DataFrame:
    """The Excel reference rows in this folder: one per 2026 market date, every value computed by Excel."""
    return pd.read_csv(HERE / "course3_excel_reference.csv", parse_dates=["date"], index_col="date",
                       float_precision="round_trip")
def test_change_since_uses_as_of_values():
    ten = read_treasury()["10 Yr"]
    expected = (ten.loc["2026-09-30"] - ten.loc["2026-09-04"]) * 100
    assert md.change_since_bp(ten, "2026-09-30", "2026-09-07") == pytest.approx(expected, abs=1e-9)
    with pytest.raises(ValueError):
        md.change_since_bp(ten, "2026-09-07", "2026-09-30")
def test_change_bp_matches_excel_reference():
    reference = read_reference()
    ten = read_treasury()["10 Yr"]
    changes = md.change_bp(ten)
    for day, row in reference.iterrows():
        assert changes.loc[day] == pytest.approx(row["chg_10y_bp"], abs=1e-9), day
        month_back = day - pd.DateOffset(months=1)
        assert md.change_since_bp(ten, day, month_back) == pytest.approx(row["chg1m_10y_bp"], abs=1e-9), day
```

```
.........................   [100%]
25 passed in 0.50s
```

"iterrows()" hands over each row in turn: its date, then its values.

Every move in the morning note is in bp, from as-of values at both ends, equal to Excel on every 2026 row. The module is done for the day: commit it.

6.6 Which notebook wrote each line: git blame

Read the Diff, Commit

```
!git -C "{work}" diff --stat
subject = 'Notebook 06: change_bp, return_pct, and change_since_bp'
!git -C "{work}" add marketdata.py test_marketdata.py
!git -C "{work}" commit -q -m "{subject}"
!git -C "{work}" log --oneline --decorate
 marketdata.py   | 53 +++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 test_marketdata.py | 51 +++++++++++++++++++++++++++++++++++++++++++++++++++
 2 files changed, 104 insertions(+)
1f5838a (HEAD -> main) Notebook 06: change_bp, return_pct, and change_since_bp
ace289a (tag: week1) Notebook 05: write_cache, read_cache, and cached_fred_series
9f01b1c Notebook 04: last_per_period and value_as_of
a439502 Notebook 03: missing_weekdays, align, and stale_runs
ceea769 Notebook 02: fred_params, parse_fred_observations, and fred_series
a28686d Notebook 01: get_secret, a .gitignore, and .env.example
```

Three Functions, Three Notebooks

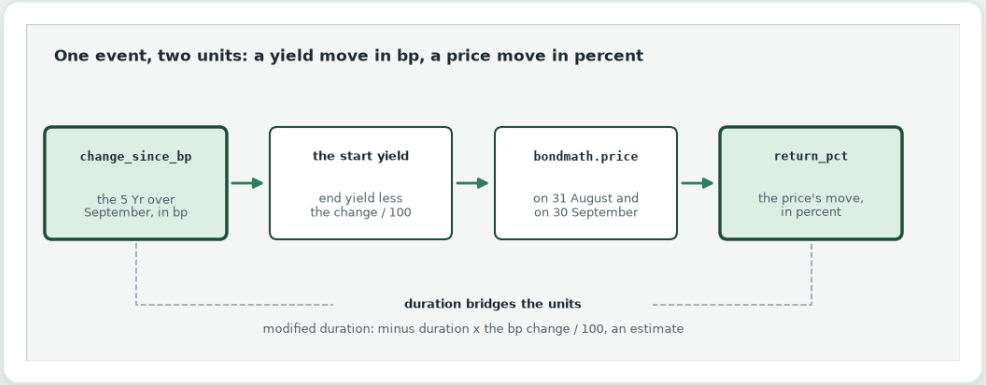
Q. Which commit wrote the first line of "get_secret", of "value_as_of", and of "change_bp"?

Each line starts with the hash of the commit that wrote it, then its line number: match them to the log.

```
lines = '-L "/def get_secret/,+1" -L "/def value_as_of/,+1" -L "/def change_bp/,+1"'
!git -C "{work}" blame -s --root {lines} marketdata.py
a28686d8  17) def get_secret(name: str, env_file: str | Path) -> str:
9f01b1c1 251) def value_as_of(series: pd.Series, as_of) -> tuple[pd.Timestamp, float]:
1f5838a2 339) def change_bp(series: pd.Series, periods: int = 1, percent_units: bool = True) -> pd.Series:
```

6.7 A price return from your bondmath

Quorvane Energy, Course 2's synthetic bond: a 5.25 percent coupon, maturing 2031-09-30, yielding 5.40 percent on 2026-09-30. Take its yield to have moved over September by exactly the 5 Yr's change.



September's Move in the 5 Yr

```
coupon_pct, maturity, yield_end_pct = 5.25, date(2031, 9, 30), 5.40
five_sept_bp = md.change_since_bp(treasury['5 Yr'], '2026-09-30', '2026-08-31')
yield_start_pct = yield_end_pct - five_sept_bp / 100
print(f'5 Yr over September: {five_sept_bp:+.0f} bp; the bond from {yield_start_pct:.2f} percent')
5 Yr over September: +60 bp; the bond from 4.80 percent
```

Price It on Both Dates

```
bond_prices = pd.Series([bondmath.price(date(2026, 8, 31), maturity, coupon_pct, yield_start_pct),
                         bondmath.price(date(2026, 9, 30), maturity, coupon_pct, yield_end_pct)],
                        index=pd.to_datetime(['2026-08-31', '2026-09-30']))
bond_return = md.return_pct(bond_prices).iloc[-1]
print(f'{bond_prices.iloc[0]:.6f} to {bond_prices.iloc[1]:.6f}: {bond_return:.4f} percent')
102.004270 to 99.350327: -2.6018 percent
```

A clean-price return: the coupon paid on 2026-09-30 and the accrued interest are left out.

Excel's PRICE, and duration bridges the units

Excel side by side. 4.800000000000001 in A1 and 5.4 in A2.

Job	Python	Excel	Result
Price on 31 August, B1	`bondmath.price(...)`	`=PRICE(DATE(2026,8,31),DATE(2031,9,30),5.25/100,A1/100,100,2,0)`	102.00426950087292
Price on 30 September, B2	`bondmath.price(...)`	`=PRICE(DATE(2026,9,30),DATE(2031,9,30),5.25/100,A2/100,100,2,0)`	99.35032727691544
The return in percent	`md.return_pct(bond_prices)`	`=(B2/B1-1)*100`	-2.601795235575677

Duration Bridges the Units

```
duration = bondmath.modified_duration(date(2026, 8, 31), maturity, coupon_pct, yield_start_pct)
estimate_pct = -duration * five_sept_bp / 100
print(f'Duration {duration:.4f}: estimate {estimate_pct:.4f} percent, repriced {bond_return:.4f}')
Duration 4.3355: estimate -2.6013 percent, repriced -2.6018
```

The two agree so closely because three small pieces nearly cancel. Convexity adds about 0.04 percent, and a month less to maturity about 0.01. Duration's percent is of the dirty price, and this return's of the clean price, which takes about 0.05 away.

One event, two units: +60 bp on the yield, -2.60 percent on the price, and duration turns one into the other only because the move is in bp.

6.8 The largest daily moves of 2026

The note's last ask.

The Biggest Days

```
ten_2026 = md.change_bp(ten).loc['2026']
rounded = ten_2026.round(6)
largest = rounded.abs().nlargest(5, keep='all')
rounded[largest.index]
date
2026-09-23   15.0
2026-03-20   14.0
2026-05-15   12.0
2026-09-10   12.0
2026-05-20   -10.0
2026-06-11   -10.0
Name: 10 Yr, dtype: float64
```

Six days: two tie at 10 bp, the fifth place.

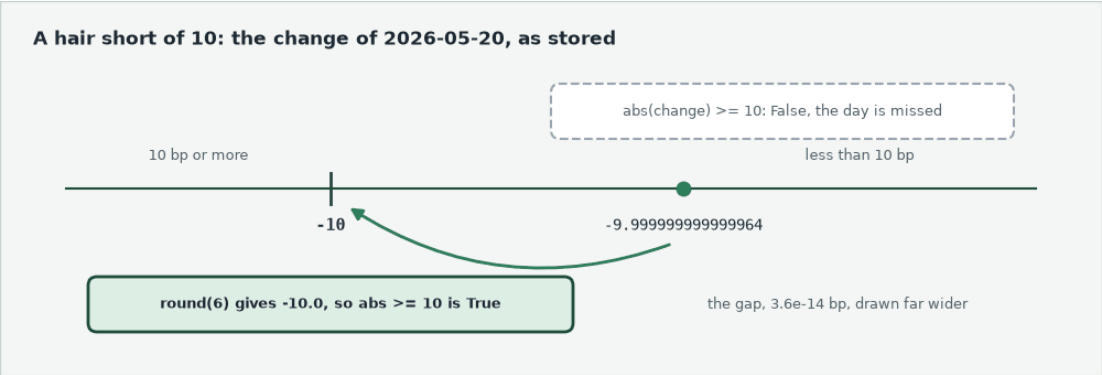
Why Round First

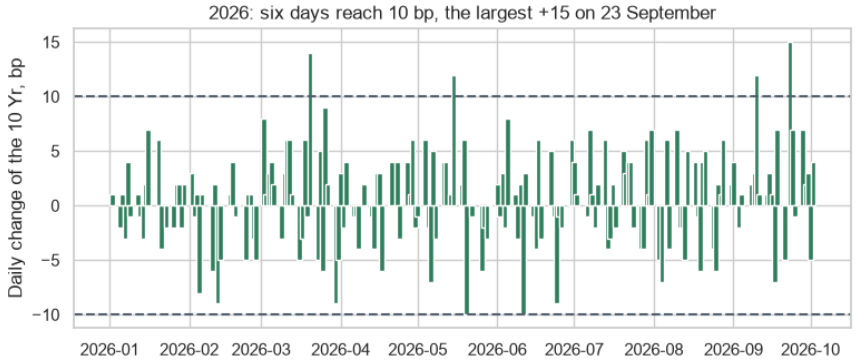
```
print(f'Unrounded: {(ten_2026.abs() >= 10).sum()} days; '
      f'rounded: {(rounded.abs() >= 10).sum()} days')
print(ten_2026.loc[['2026-05-20', '2026-06-11']].tolist())
Unrounded: 4 days; rounded: 6 days
[-9.999999999999964, -9.999999999999964]
```

Excel side by side. The daily changes in C251:C440.

Job	Python	Excel	Result
The largest rise, and its date	`ten_2026.max(), .idxmax()`	`=MAX(C251:C440), =XLOOKUP(MAX(C251:C440),C251:C440,A251:A440)`	15.000000000000036 on 2026-09-23
The largest fall, and its date	`ten_2026.min(), .idxmin()`	`=MIN(C251:C440), the same XLOOKUP`	-9.999999999999964 on 2026-05-20, the first of two

A hair short of 10, and the year day by day





Both falls are stored a hair short of 10: the binary tail of Course 1's "0.1 + 0.2". Round to 6 decimals before a count, a threshold or a rank.

Excel side by side.

Job	Python	Excel	Result
Days of 10 bp or more	`(ten_2026.abs() >= 10).sum()`	`=SUMPRODUCT(--(ABS(C251:C440)>=10))`	4
The same, rounded	`(rounded.abs() >= 10).sum()`	`=SUMPRODUCT(--(ROUND(ABS(C251:C440),6)>=10))`	6
Days of +1 bp	`((rounded >= 0.5) & (rounded < 1.5)).sum()`	`=COUNTIFS(C251:C440,">=0.5",C251:C440,"<1.5")`	21

2026's biggest 10 Yr day was +15 bp on 23 September. A count or a rank of computed changes rounds the binary tail off first. The note is answered.

Bring it home, and what you can now do

Copy the two files and Excel's reference rows into your own folder, test, and commit. In a PowerShell terminal in VS Code, with the course's ".venv" active:

```
$work = "$env:TEMP\pfi_course3_06_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py", "$work\course3_excel_reference.csv" .
python -m pytest -q test_marketdata.py
git diff --stat
git add marketdata.py test_marketdata.py
git commit -m "Notebook 06: change_bp, return_pct, and change_since_bp"
git blame -L "/def change_bp/,+1" marketdata.py
```

The copy replaces your two files, so anything of your own goes into the work folder first. pytest shows 25 passed, and "git blame" names your new commit for the first line of "change_bp". The reference file stays out of Git. In Colab, download the three files from the work folder under "/tmp" instead.

You Can Now

Say why yields and spreads change in bp and prices in percent.

Measure a change k rows back with "change_bp", for a yield in percent or a spread in bp.

Explain why a percent change of a yield misleads.

Measure a change since a date with "change_since_bp", from as-of values at both ends.

Price a bond on two dates with "bondmath" and put its return beside the yield change.

Round before a count or a rank of computed changes.

Find the commit behind a line with "git blame -L".

Predict first: rows back, a percent, the biggest day

Predict 1

"chg_5_bp" on Tuesday 2026-09-08 goes back five rows. From which date?

A) Tuesday 2026-09-01

B) Monday 2026-08-31

C) Thursday 2026-09-03

```
tuesday = ten.index.get_loc(pd.Timestamp('2026-09-08'))
print(f'Five rows back from 2026-09-08: {ten.index[tuesday - 5]:%Y-%m-%d}')
```

Predict 2

A yield moves from 4.00 to 4.10 percent, 10 bp. What does "=B2/B1-1" show in Excel's percent format? Write down a number.

```
pair = pd.Series([4.00, 4.10])
print(f'{(pair.iloc[1] - pair.iloc[0]) * 100:.1f} bp')
print(f'{pair.pct_change().iloc[1] * 100:.1f} percent of the yield')
```

Predict 3

Over the whole file, is the 10 Yr's largest daily percent change on the same day as its largest move in bp?

A) The same day

B) A different day, from a lower starting yield

C) A different day, from a higher starting yield

```
all_changes = pd.DataFrame({'from_pct': ten.shift(1), 'chg_bp': md.change_bp(ten),
                            'pct_of_yield': ten.pct_change() * 100})
biggest = [all_changes['chg_bp'].abs().idxmax(), all_changes['pct_of_yield'].abs().idxmax()]
all_changes.loc[biggest].round(2)
```

Predict first: a line's commit, and the tail at 5 bp

Predict 4

"_check_series" was written in notebook 04, and "change_bp" calls it. Which commit does "git blame" name for the line "_check_series(series, "series")" inside "change_bp"?

A) Notebook 04's

B) Today's

C) The first commit

```
!git -C "{work}" blame -s -L "/def change_bp/,/return difference/" marketdata.py
```

Predict 5

At 10 bp the tail hid 2 days. At 5 bp, how many does it hide?

A) None: only large moves carry it

B) About 2

C) More than 10

```
print(f'At 5 bp: {(ten_2026.abs() >= 5).sum()} days unrounded, '
      f'{(rounded.abs() >= 5).sum()} rounded')
```

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

Docstring

Units, conventions, and what it must never do: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: "series.pct_change() * 100" differs from Excel's daily change in bp by up to 11.98 bp.

Code that runs is not the same as code that is right.

The note, answered

The note asked	Answer
On the day, week, month and year	10 Yr +4, +11, +49 and +110 bp as of 2026-10-02, with the 2, 5 and 30 Yr and both spreads beside it
Each in its right unit, tied to Excel	bp for yields and spreads, percent for prices; 190 of 190 rows equal
A holiday never a zero move	Tuesday 8 September is a change from Friday 4 September
September's 5 Yr move on a bond	+60 bp, and Quorvane's clean price -2.60 percent
The biggest days of 2026	+15 bp on 2026-09-23; -10 bp on 2026-05-20 and 2026-06-11

Be proud: two mistakes that raise no error, a wrong unit and a hidden binary tail, are now caught in code.

Next, Rolling Windows and Volatility (notebook 07) turns today's daily changes into a measure of how much a series has been moving.